Name: Abhishek Kamlesh Vishwakarma
Class: D20C **Roll.No:** 35

Experiment No.3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.
Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
4. Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (5001, 5002, 5003).
Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the Longest Chain Rule:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A reward transaction is added automatically to pay the miner.

6. Chain Replacement

When /replace_chain is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

- **Python 3.x**
- **Flask** – Web framework for API endpoints

pip install Flask

- **Requests** – For HTTP communication between nodes

pip install requests==2.18.4

- **Postman** – For testing API requests
- Python Standard Libraries:
    - datetime
    - jsonify
    - hashlib
    - uuid4
    - urlparse
    - request

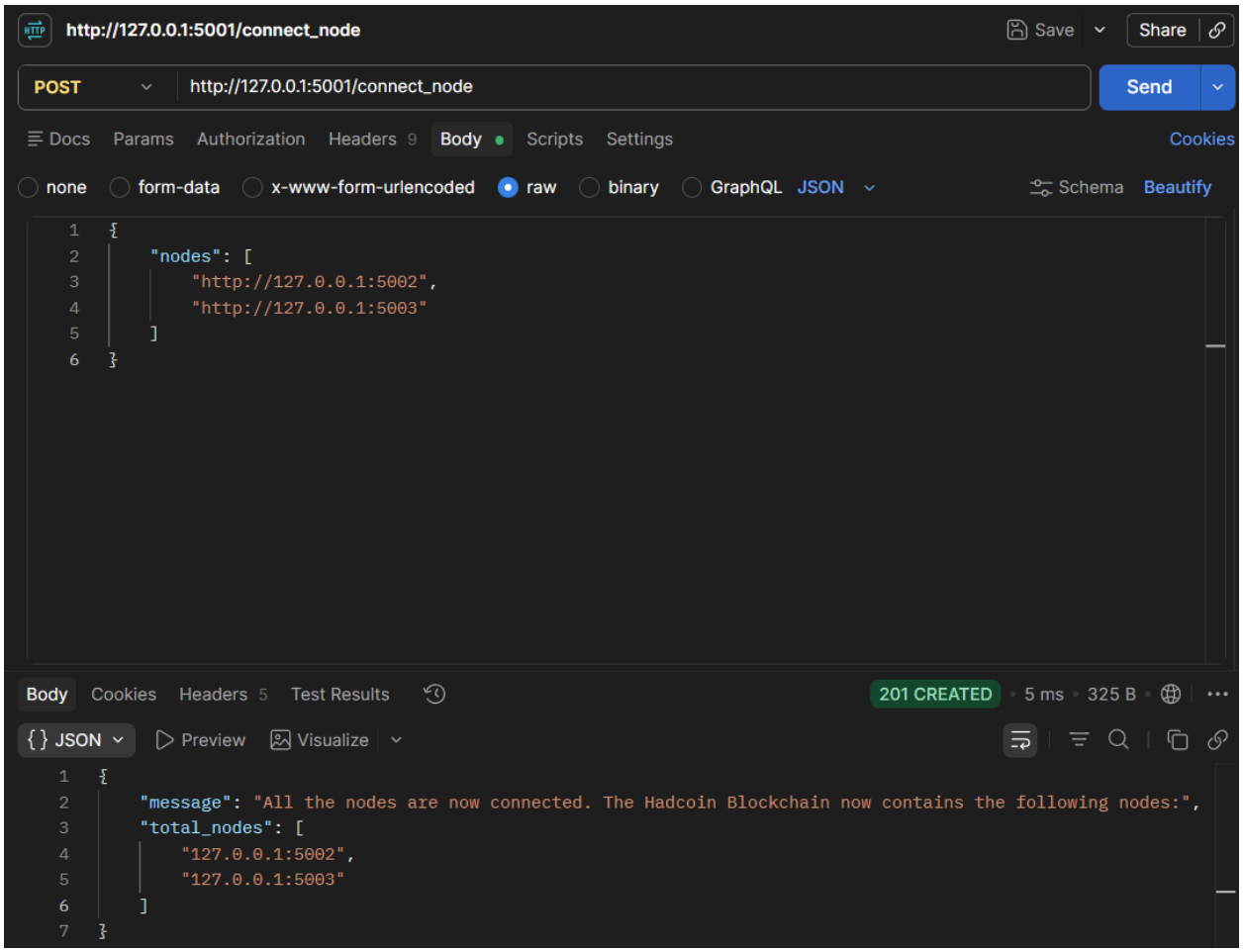
Procedure and Output:

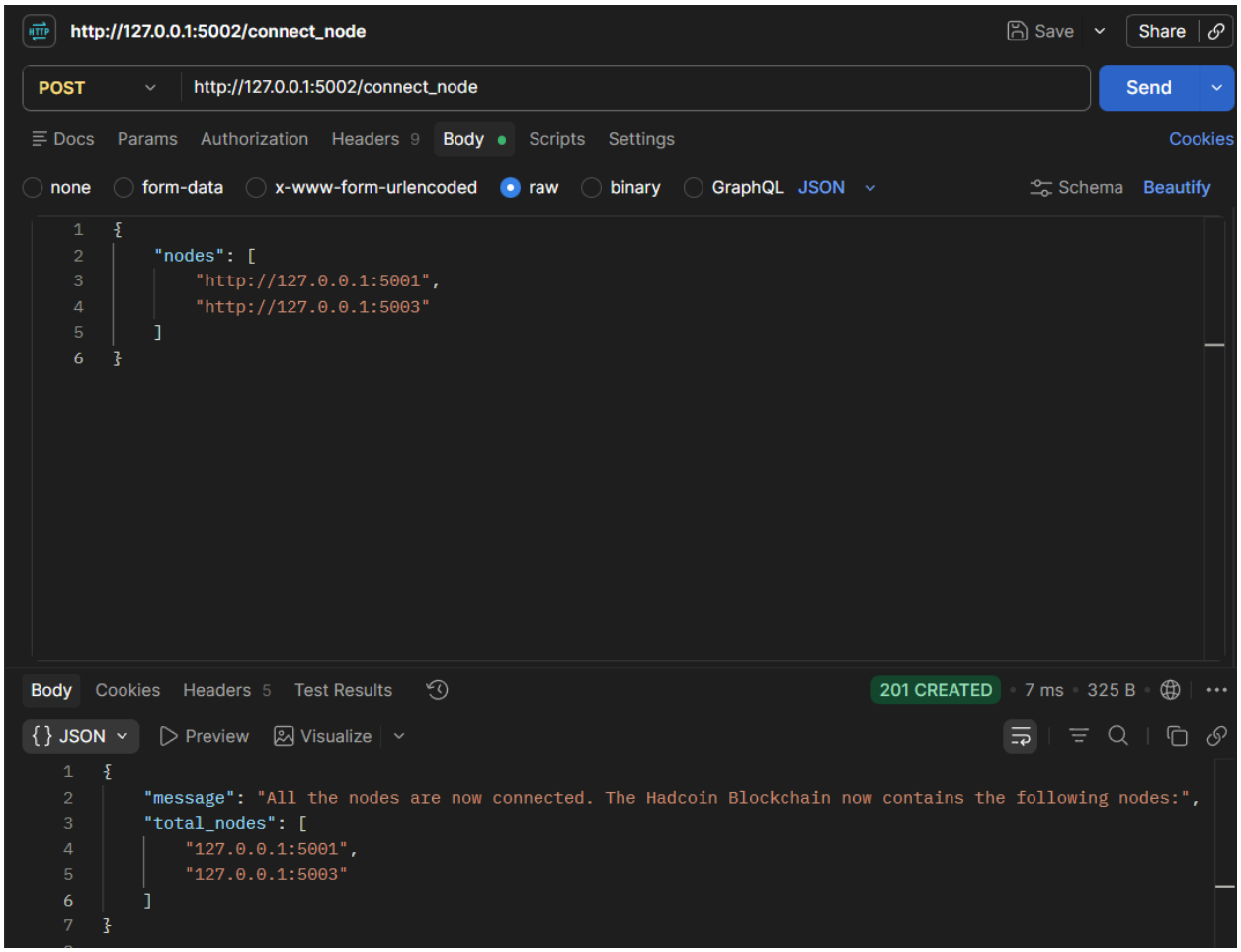
1. Download hadcoin_node_5001.py, hadcoin_node_5002.py, hadcoin_node_5003.py.
2. Install required packages.
3. Run each node in separate terminals.

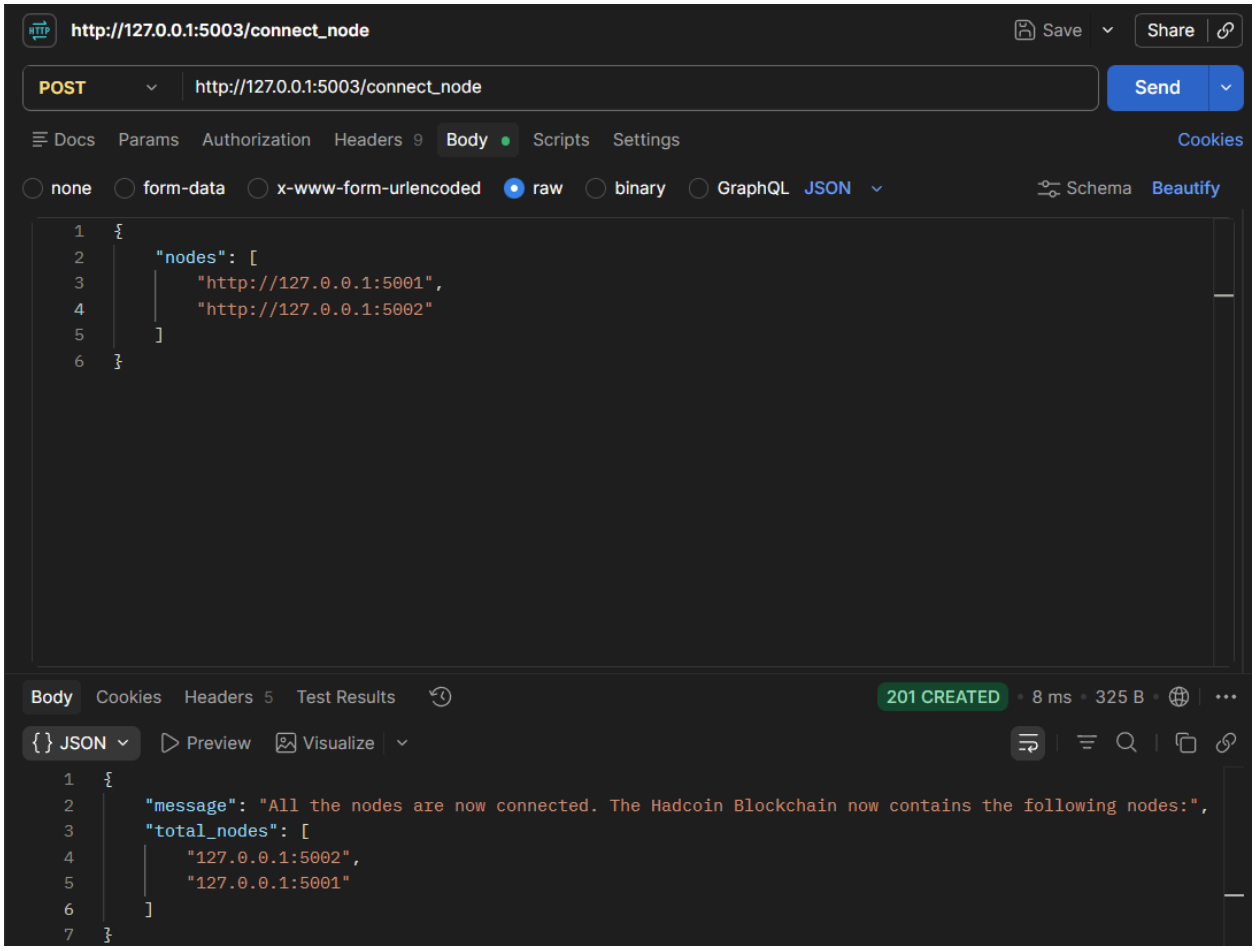
```
127.0.0.1 - - [25/Aug/2026 09:45:18] "GET / HTTP/1.1" 404 -
(venv) PS D:\SEM VII\BC Exps\Exp 3> python hadcoin_node_5001.py
 * Serving Flask app 'hadcoin_node_5001'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5001
 * Running on http://192.168.173.139:5001
Press CTRL+C to quit
127.0.0.1 - - [25/Aug/2026 09:56:49] "POST /connect_node HTTP/1.1" 400 -
127.0.0.1 - - [25/Aug/2026 09:57:29] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [25/Aug/2026 09:57:43] "GET /connect_node HTTP/1.1" 405 -
127.0.0.1 - - [25/Aug/2026 10:00:34] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [25/Aug/2026 10:03:13] "POST /get_chain HTTP/1.1" 405 -
127.0.0.1 - - [25/Aug/2026 10:03:23] "POST /get_chain HTTP/1.1" 405 -
127.0.0.1 - - [25/Aug/2026 10:03:48] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:05:27] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [25/Aug/2026 10:06:17] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [25/Aug/2026 10:07:18] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:07:38] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:07:41] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:07:57] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:09:38] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:10:33] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:13:05] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:16:24] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:17:45] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:18:16] "GET /is_valid HTTP/1.1" 200 -
```

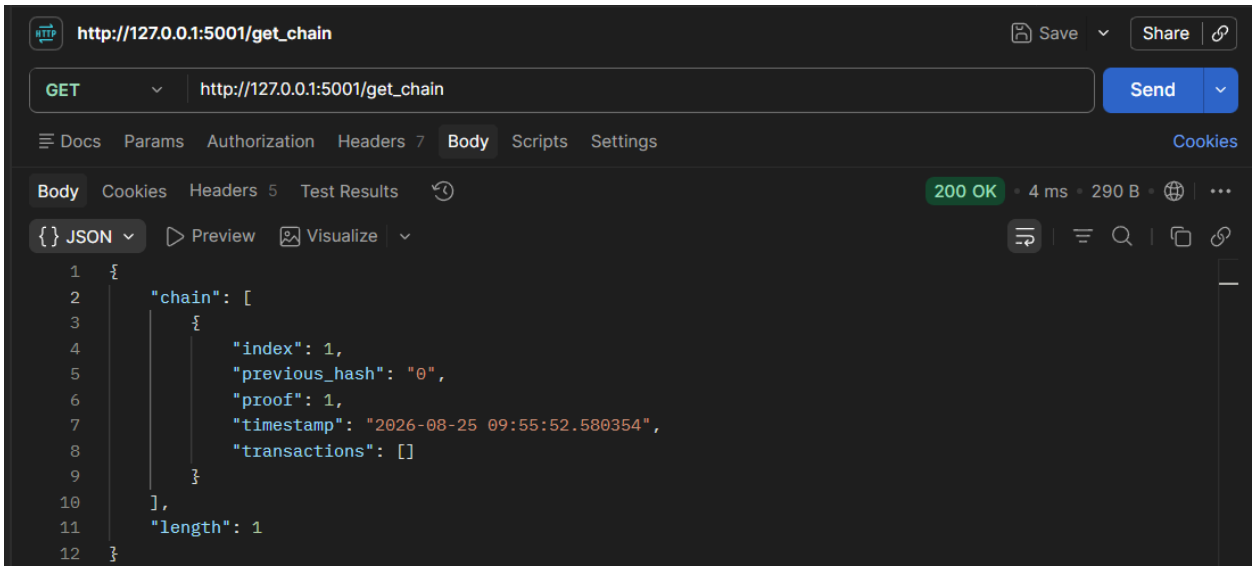
```
 * Running on http://127.0.0.1:5002
(venv) PS D:\SEM VII\BC Exps\Exp 3> python hadcoin_node_5002.py
(venv) PS D:\SEM VII\BC Exps\Exp 3> python hadcoin_node_5002.py
 * Serving Flask app 'hadcoin_node_5002'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5002
 * Running on http://192.168.173.139:5002
Press CTRL+C to quit
127.0.0.1 - - [25/Aug/2026 10:01:42] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [25/Aug/2026 10:11:40] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:13:05] "GET /replace_chain HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:16:24] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:17:49] "GET /get_chain HTTP/1.1" 200 -
```

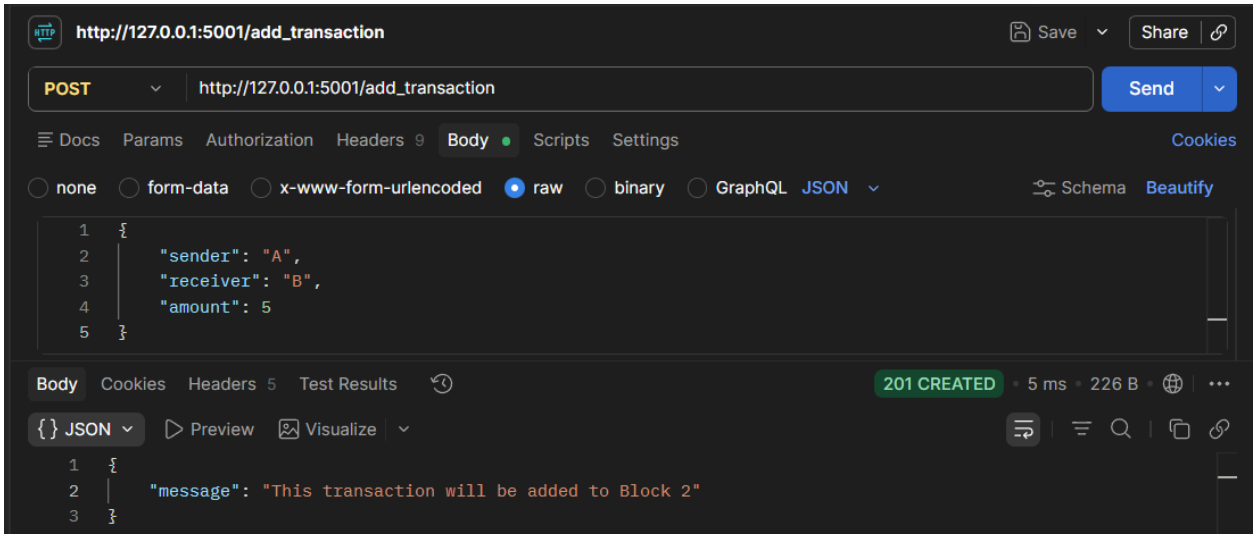
```
Press CTRL+C to quit
(venv) PS D:\SEM VII\BC Exps\Exp 3> python hadcoin_node_5003.py
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5003
 * Running on http://192.168.173.139:5003
Press CTRL+C to quit
127.0.0.1 - - [25/Aug/2026 10:02:13] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [25/Aug/2026 10:12:22] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:13:05] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:16:24] "GET /replace_chain HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 10:17:54] "GET /get_chain HTTP/1.1" 200 -
```

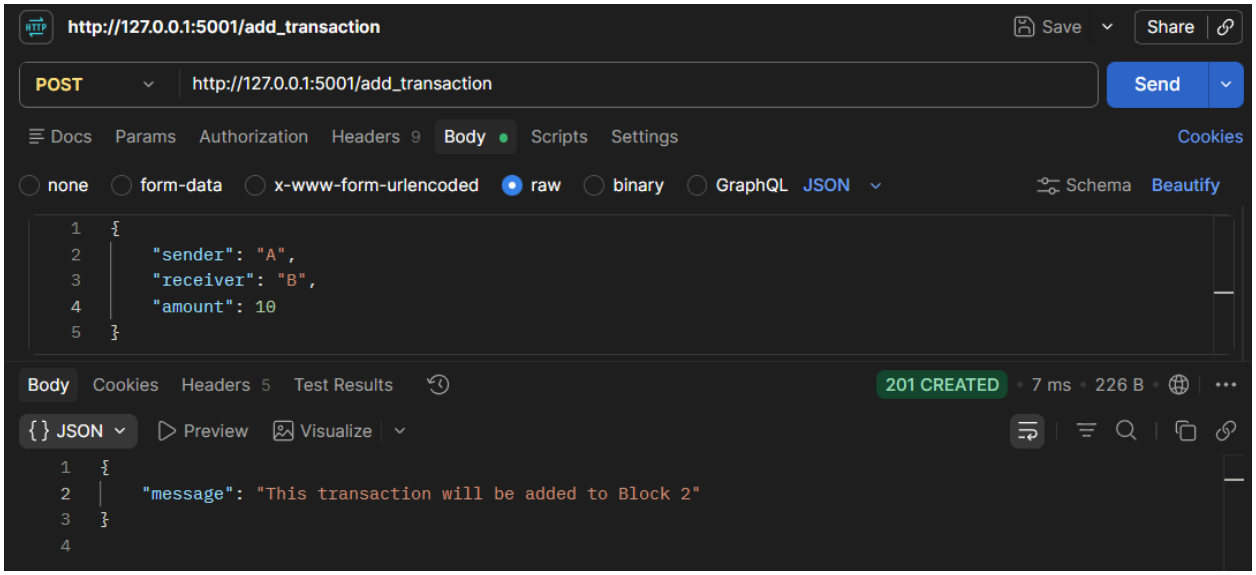

http://127.0.0.1:5001/connect_node
Save
Share
POST
http://127.0.0.1:5001/connect_node
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"nodes": [
"http://127.0.0.1:5002",
"http://127.0.0.1:5003"
]
}
Body
Cookies
Headers 5
Test Results
201 CREATED
5 ms
325 B
{ } JSON
Preview
Visualize
{
"message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
"total_nodes": [
"127.0.0.1:5002",
"127.0.0.1:5003"
]
}

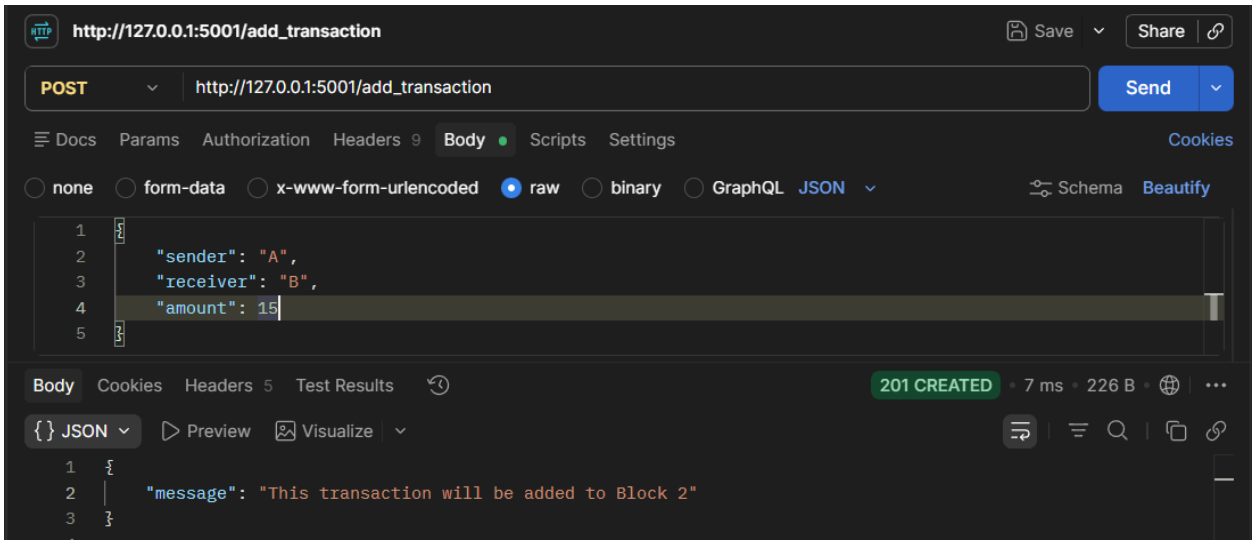

http://127.0.0.1:5002/connect_node
Save
Share
POST
http://127.0.0.1:5002/connect_node
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"nodes": [
"http://127.0.0.1:5001",
"http://127.0.0.1:5003"
]
}
Body
Cookies
Headers 5
Test Results
201 CREATED
7 ms
325 B
{} JSON
Preview
Visualize
{
"message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
"total_nodes": [
"127.0.0.1:5001",
"127.0.0.1:5003"
]
}

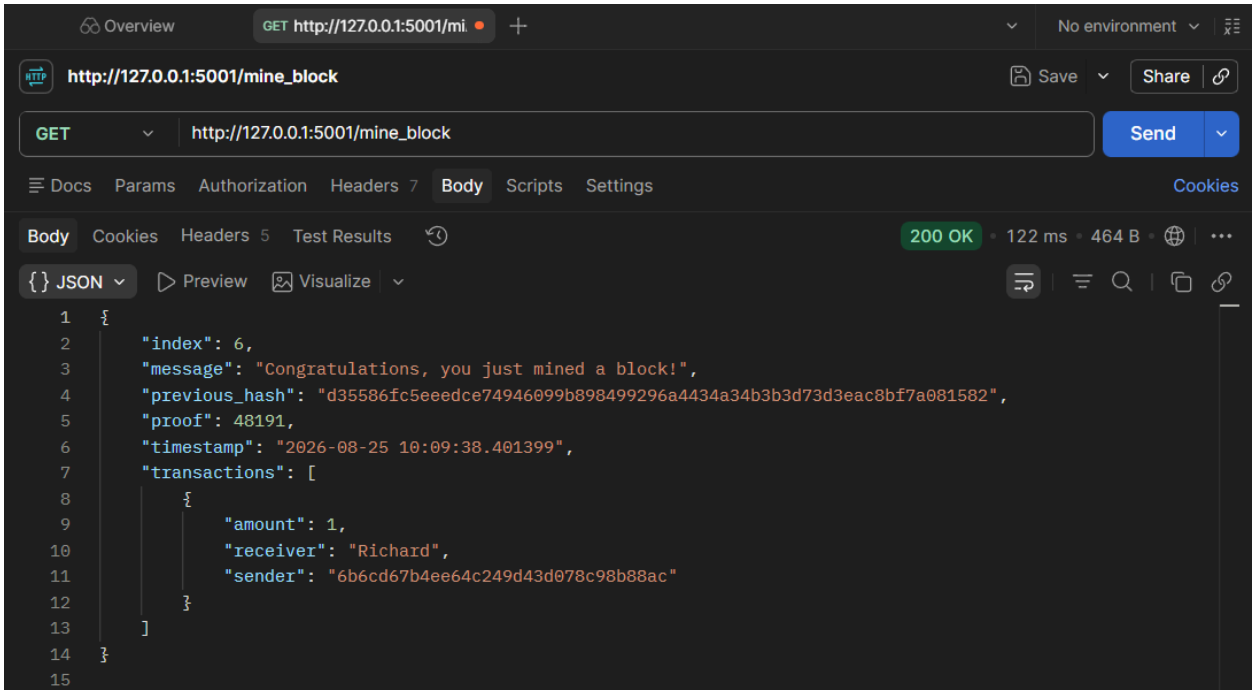

http://127.0.0.1:5003/connect_node
Save
Share
POST
http://127.0.0.1:5003/connect_node
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"nodes": [
"http://127.0.0.1:5001",
"http://127.0.0.1:5002"
]
}
Body Cookies Headers 5 Test Results
201 CREATED 8 ms 325 B
{ } JSON Preview Visualize
{
"message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
"total_nodes": [
"127.0.0.1:5002",
"127.0.0.1:5001"
]
}

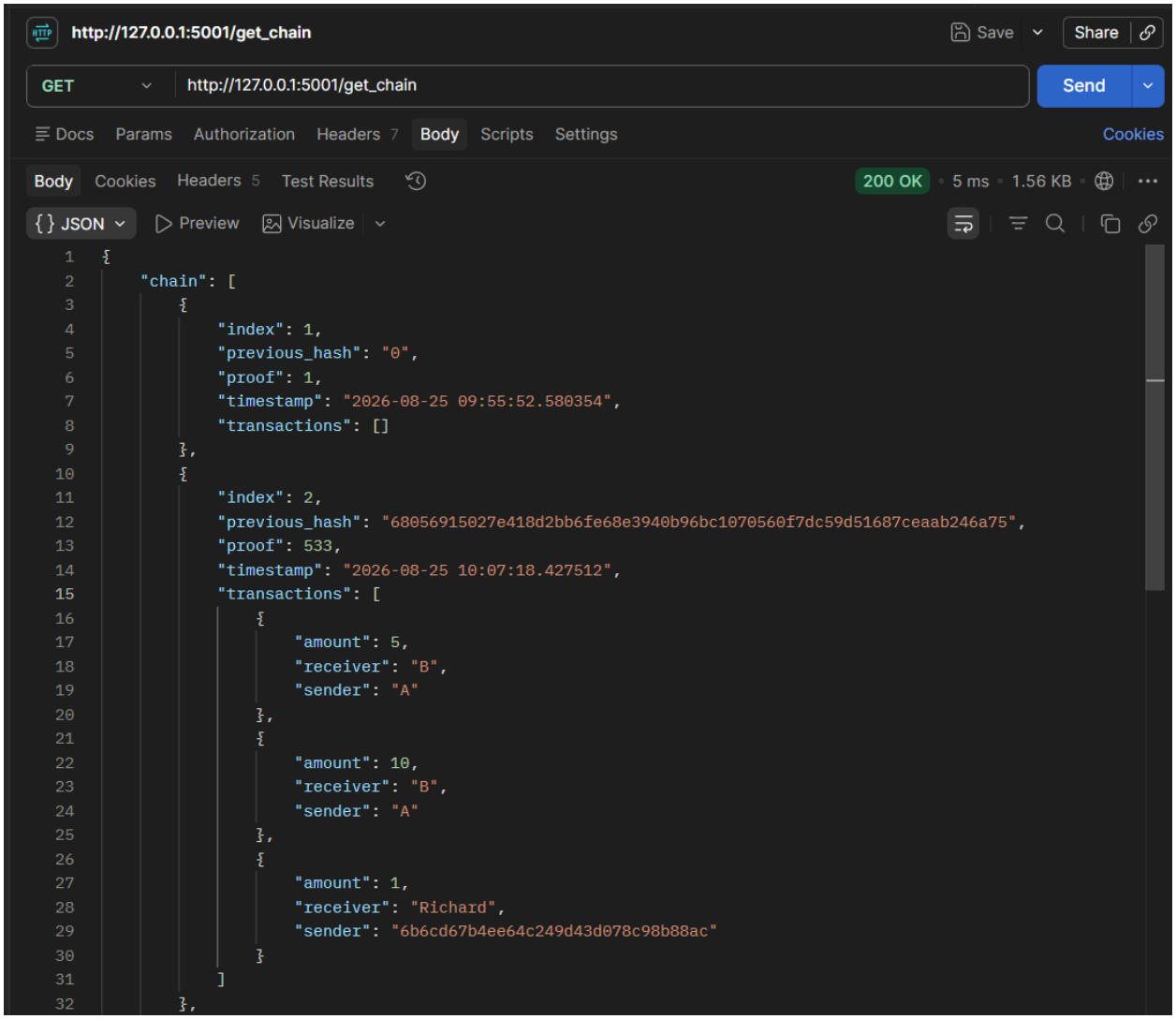

http://127.0.0.1:5001/get_chain
Save
Share
GET
http://127.0.0.1:5001/get_chain
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Cookies
Body Cookies Headers 5 Test Results
200 OK 4 ms 290 B
{ } JSON Preview Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-25 09:55:52.580354",
"transactions": []
}
],
"length": 1
}

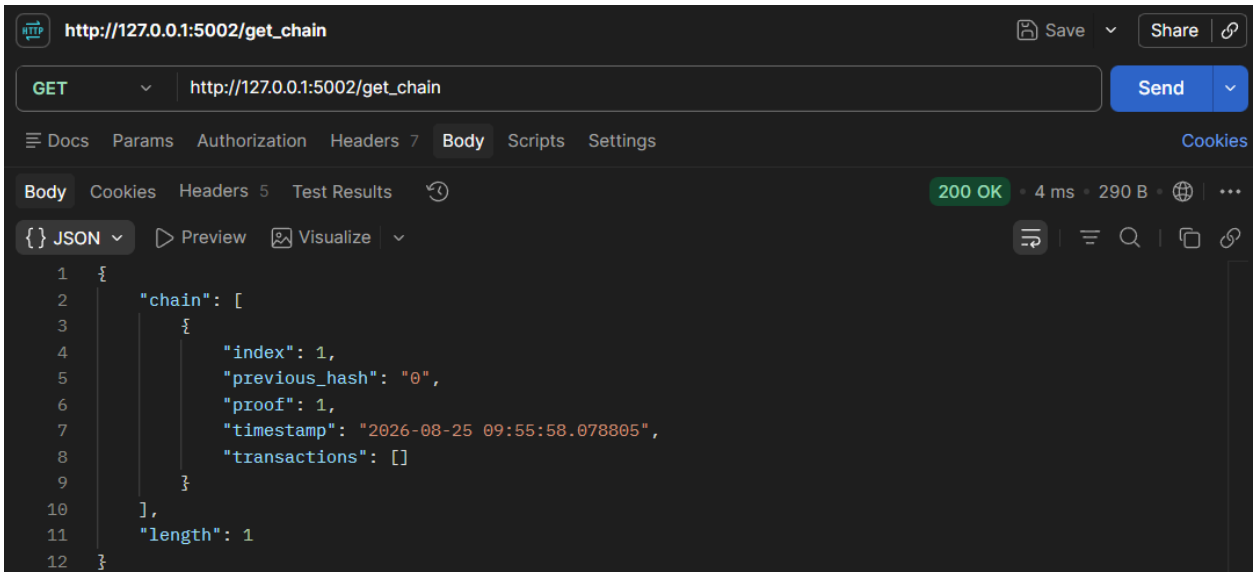

http://127.0.0.1:5001/add_transaction
Save
Share
POST
http://127.0.0.1:5001/add_transaction
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"sender": "A",
"receiver": "B",
"amount": 5
}
Body Cookies Headers 5 Test Results
201 CREATED 5 ms 226 B
{ } JSON Preview Visualize
{
"message": "This transaction will be added to Block 2"
}

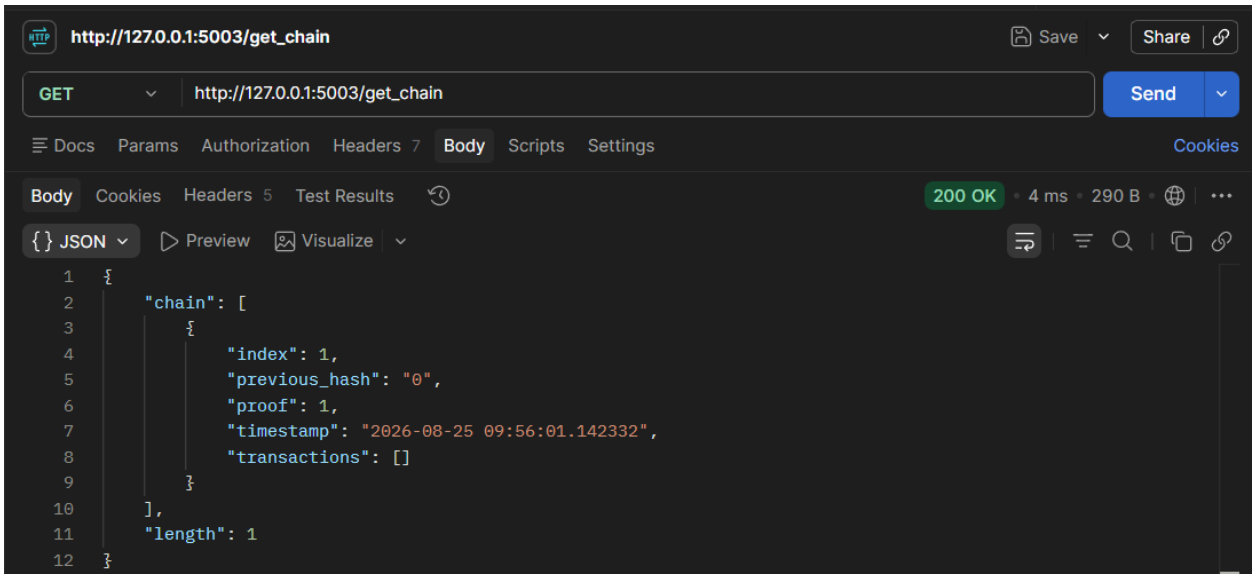

http://127.0.0.1:5001/add_transaction
Save
Share
POST
http://127.0.0.1:5001/add_transaction
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"sender": "A",
"receiver": "B",
"amount": 10
}
Body Cookies Headers 5 Test Results
201 CREATED 7 ms 226 B
{ } JSON Preview Visualize
{
"message": "This transaction will be added to Block 2"
}

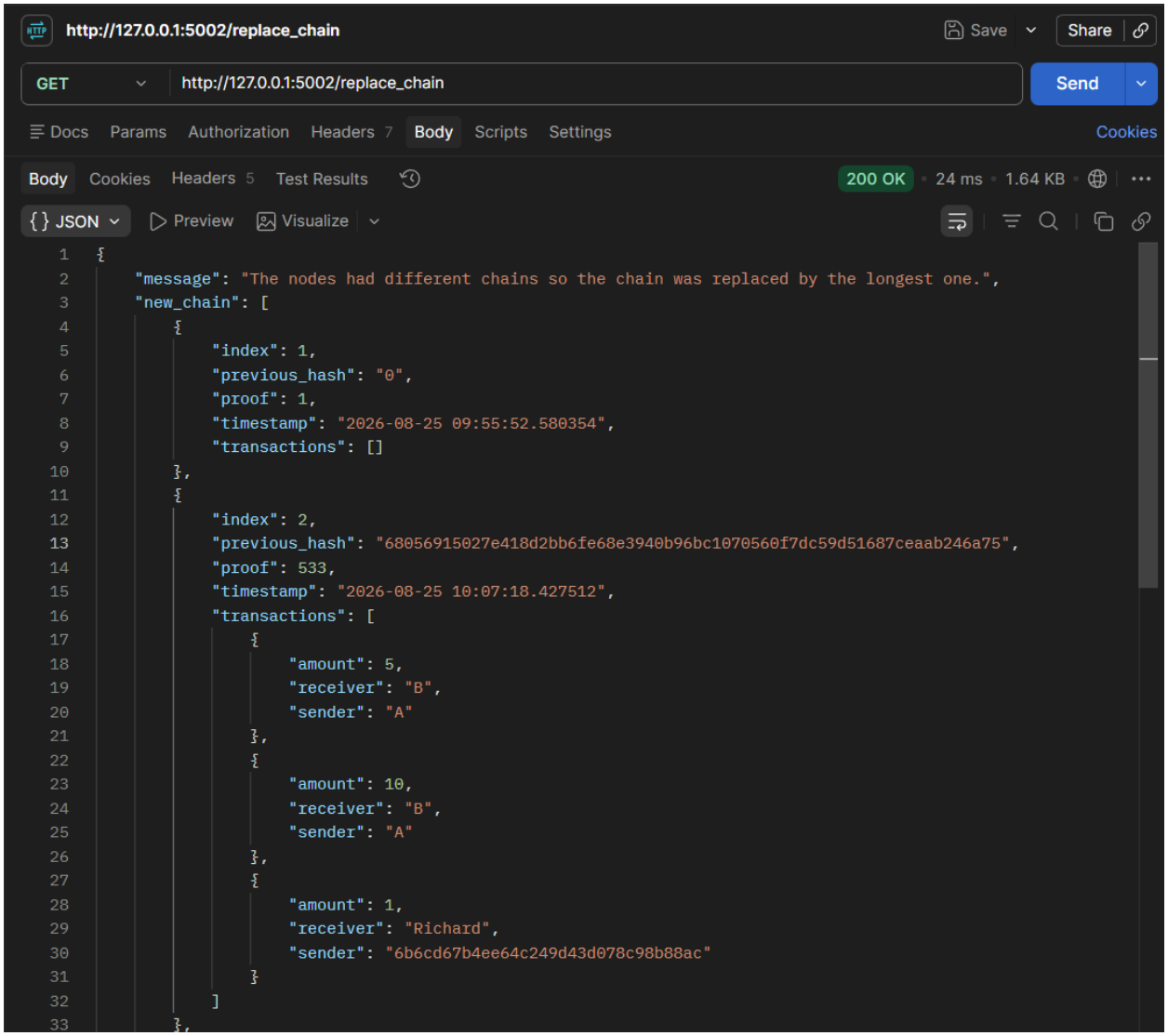

http://127.0.0.1:5001/add_transaction
Save
Share
POST
http://127.0.0.1:5001/add_transaction
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"sender": "A",
"receiver": "B",
"amount": 15
}
Body Cookies Headers 5 Test Results
201 CREATED 7 ms 226 B
{ } JSON Preview Visualize
{
"message": "This transaction will be added to Block 2"
}

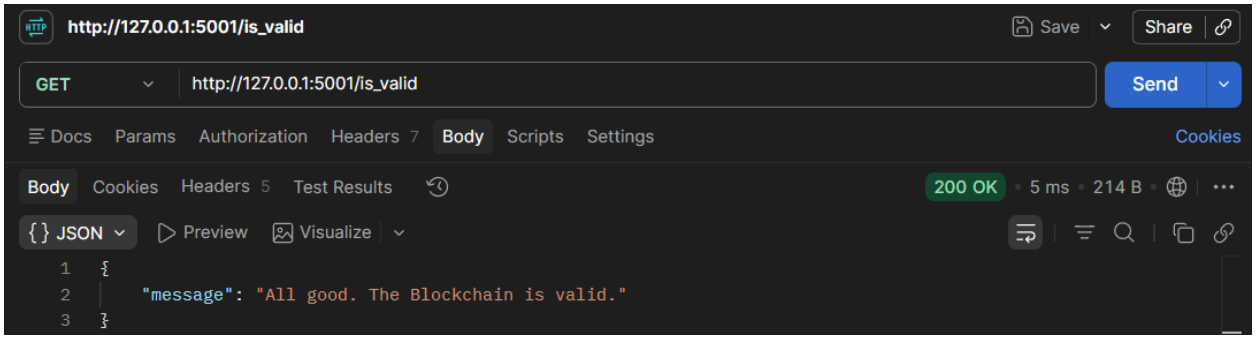

Overview
GET http://127.0.0.1:5001/mi
No environment
http://127.0.0.1:5001/mine_block
Save
Share
GET
http://127.0.0.1:5001/mine_block
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
Body
Cookies
Headers 5
Test Results
200 OK
122 ms
464 B
{ } JSON
Preview
Visualize
{
"index": 6,
"message": "Congratulations, you just mined a block!",
"previous_hash": "d35586fc5eeedce74946099b898499296a4434a34b3b3d73d3eac8bf7a081582",
"proof": 48191,
"timestamp": "2026-08-25 10:09:38.401399",
"transactions": [
{
"amount": 1,
"receiver": "Richard",
"sender": "6b6cd67b4ee64c249d43d078c98b88ac"
}
]
}


http://127.0.0.1:5001/get_chain
Save
Share
GET
http://127.0.0.1:5001/get_chain
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Cookies
Body Cookies Headers 5 Test Results
200 OK 5 ms 1.56 KB
{ } JSON Preview Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-25 09:55:52.580354",
"transactions": []
},
{
"index": 2,
"previous_hash": "68056915027e418d2bb6fe68e3940b96bc1070560f7dc59d51687ceaab246a75",
"proof": 533,
"timestamp": "2026-08-25 10:07:18.427512",
"transactions": [
{
"amount": 5,
"receiver": "B",
"sender": "A"
},
{
"amount": 10,
"receiver": "B",
"sender": "A"
},
{
"amount": 1,
"receiver": "Richard",
"sender": "6b6cd67b4ee64c249d43d078c98b88ac"
}
]
},


http://127.0.0.1:5002/get_chain
Save
Share
GET
http://127.0.0.1:5002/get_chain
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Cookies
Body Cookies Headers 5 Test Results
200 OK 4 ms 290 B
{ } JSON Preview Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-25 09:55:58.078805",
"transactions": []
}
],
"length": 1
}


http://127.0.0.1:5003/get_chain
Save
Share
GET
http://127.0.0.1:5003/get_chain
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
Body
Cookies
Headers 5
Test Results
200 OK
4 ms
290 B
{ } JSON
Preview
Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-25 09:56:01.142332",
"transactions": []
}
],
"length": 1
}





Conclusion: A custom cryptocurrency and a three-node simulated blockchain network were successfully built using Python. Essential concepts such as Proof-of-Work mining, transaction validation, and decentralized consensus via the Longest Chain Rule were practically implemented.